

MODULE 1

DATA PIPELINES AND VISUALISATION

MLFP — ML Foundations for Professionals — Terrene Open Academy

"Can you trust a number you didn't explore yourself?"

WHAT YOU WILL LEARN

BY THE END OF THIS MODULE

- **Write Python** from scratch — variables, functions, loops, conditionals
- **Load and explore data** with Polars (not pandas, not Excel)
- **Filter, join, and aggregate** millions of rows in seconds
- **Visualise patterns** with Plotly and ModelVisualizer
- **Profile data automatically** with DataExplorer
- **Build end-to-end pipelines** with PreprocessingPipeline

THREE DEPTH LAYERS

FOUNDATIONS — Everyone follows. No maths, no jargon.

THEORY — The maths behind the intuition. Follow if you can.

ADVANCED — Cutting-edge insights for experts.

YOUR JOURNEY: 8 LESSONS

#	Lesson	You Will Be Able To...
1.1	Your First Data Exploration	Load a CSV and inspect shape, columns, statistics
1.2	Filtering and Transforming Data	Slice data by any condition, create computed columns
1.3	Functions and Aggregation	Write reusable code, compute group statistics
1.4	Joins and Multi-Table Data	Combine datasets (HDB + MRT + schools)
1.5	Window Functions and Trends	Compute rolling averages and year-on-year changes
1.6	Data Visualisation	Build interactive charts with Plotly / ModelVisualizer
1.7	Automated Data Profiling	Profile an entire dataset in one call with DataExplorer
1.8	Data Pipelines and End-to-End Project	Build a full EDA pipeline: load, profile, clean, visualise

KAILASH ENGINES YOU WILL MEET

DATAEXPLORER

Automated profiling: distributions, correlations, 8 alert types — one call.

MODELVISUALIZER

Histograms, scatter plots, bar charts, heatmaps — interactive EDA charts.

PREPROCESSINGPIPELINE

Auto-detect types, encode categoricals, scale numerics, impute missing values.

Why Kailash? Instead of writing 50 lines of boilerplate, you call one engine. The pattern repeats across all MLFP modules: learn the concept, then let the engine handle the plumbing.

TOOLS AND SETUP

WHAT YOU NEED

- **Python 3.11+** via `uv`
- **Polars** — lightning-fast DataFrames
- **Plotly** — interactive visualisation
- **Kailash ML** — `pip install kailash-ml`

```
# Quick setup
uv pip install polars plotly kailash-ml python-dotenv requests
```

THREE EXERCISE FORMATS

Format	Location
Local .py	modules/mlfp01/local/
Jupyter	modules/mlfp01/notebooks/
Google Colab	modules/mlfp01/colab/

All three formats produce identical results. Choose the one that fits your workflow.

THE STORY THAT STARTS EVERYTHING

SINGAPORE HDB RESALE FLASH CRASH — Q4 2023

- October 2023: HDB resale prices had risen 10 consecutive quarters
- Then a **sudden anomaly** — median prices in three mature estates dropped 8-12% in a single month
- Property portals and dashboards showed *nothing unusual*
- The anomaly was only caught by a data analyst who ran exploratory analysis on raw transaction data

The Dashboard Said Everything Was Fine.

Aggregated dashboards masked the signal. Monthly medians smoothed out the crash. Only row-level EDA revealed the pattern.

By the end of today, you will be able to load this exact dataset, filter for those estates, and identify the anomaly yourself.

WHAT WENT WRONG?

DASHBOARD BLINDNESS

- **Aggregation hides outliers** — a mean over 50,000 transactions masks 200 anomalous ones
- **Pre-built charts answer yesterday's questions** — they cannot ask new ones
- **No distribution view** — the dashboard showed a single number, not a histogram

WHAT EDA WOULD HAVE CAUGHT

- A **bimodal distribution** in the price column
- **Temporal clustering** — low prices all in a 2-week window
- **Flat type correlation** — only 3-room and 4-room affected

The Fix:

```
df.filter(pl.col("town") == "QUEENSTOWN")  
  .group_by("month")  
  .agg(pl.col("resale_price").median())
```

3 lines of Polars. That is all it took.

LESSON 1.1: YOUR FIRST DATA EXPLORATION

VARIABLES, DATA TYPES, AND POLARS BASICS

WHAT YOU WILL LEARN

- Assign variables and perform arithmetic
- Understand Python data types: `int`, `float`, `str`, `bool`
- Format output with `print()` and f-strings
- Load a CSV into a Polars DataFrame
- Inspect shape, columns, head, and summary statistics

No Prerequisites

This is your first lesson. No prior Python experience required.

First 90 min: Python REPL basics

Second 90 min: Load and explore real data

VARIABLES AND ASSIGNMENT

A VARIABLE IS A LABEL

A variable stores a value so you can use it later.
Think of it as a labelled box.

```
price = 550000
town = "Queenstown"
rooms = 4
is_nature = True

print(f"A {rooms}-room flat in {town} costs ${price}.")
# Output: A 4-room flat in Queenstown costs $550,000
```

FOUR CORE DATA TYPES

Type	Example	Use Case
int	550000	Counts, IDs
float	4.75	Prices, ratios
str	"Queenstown"	Names, labels
bool	True	Flags, filters

ARITHMETIC AND EXPRESSIONS

BASIC OPERATORS

```
price = 550000
area_sqm = 92

# Price per square metre
price_per_sqm = price / area_sqm
print(f"Price per sqm: ${price_per_sqm:,.0f}")
# Output: Price per sqm: $5,978

# Percentage calculation
down_payment_pct = 0.25
down_payment = price * down_payment_pct
print(f"Down payment: ${down_payment:,.0f}")
# Output: Down payment: $137,500
```

OPERATOR REFERENCE

Operator	Meaning	Example
+	Add	5 + 3 = 8
-	Subtract	10 - 4 = 6
*	Multiply	6 * 7 = 42
/	Divide	10 / 3 = 3.33
//	Floor div	10 // 3 = 3
**	Power	2 ** 10 = 1024
%	Modulo	10 % 3 = 1

STRINGS AND F-STRINGS

STRING OPERATIONS

```
town = "Ang Mo Kio"
flat_type = "4 ROOM"

# String methods
print(town.upper())      # "ANG MO KIO"
print(flat_type.lower()) # "4 room"
print(town.replace(" ", "_")) # "Ang_Mo_Kio"

# String length
print(len(town)) # 10

# Concatenation
label = town + " - " + flat_type
print(label) # "Ang Mo Kio - 4 ROOM"
```

F-STRING FORMATTING

```
price = 485000
area = 93.5
town = "Toa Payoh"

# Basic interpolation
print(f"Town: {town}")

# Number formatting
print(f"Price: ${price:,}") # $485,000
print(f"Area: {area:.1f} sqm") # 93.5 sqm
print(f"PSM: ${price/area:,.0f}") # $5,187

# Alignment (useful for reports)
print(f"{'Town':<15} {'Price':>10}")
print(f"{'Town':<15} {'price':>10,}")
```

YOUR FIRST DATAFRAME

LOADING DATA WITH POLARS

```
import polars as pl

# Load Singapore weather data
df = pl.read_csv("data/singapore_weather.csv")

# What do we have?
print(f"Shape: {df.shape}")           # (1095, 8) = 1095 rows, 8 columns
print(f"Columns: {df.columns}")       # ['date', 'station', 'rainfall_mm', ...]

# First 5 rows
print(df.head())
```

Key Insight: A DataFrame is a table. Rows are observations (one per day). Columns are variables (temperature, rainfall). This structure is the foundation of all data analysis.

INSPECTING YOUR DATA

SUMMARY STATISTICS

```
# Statistical summary of every column
print(df.describe())

# Output (abridged):
# +-----+-----+-----+
# | statistic | temp_max | rainfall |
# | ---      | ---      | ---      |
# | str       | f64       | f64       |
# +-----+-----+-----+
# | count     | 1095.0    | 1095.0    |
# | mean      | 31.4      | 6.2       |
# | std       | 1.3       | 15.8      |
# | min       | 26.1      | 0.0       |
# | max       | 36.2      | 138.4     |
# +-----+-----+-----+
```

WHAT DESCRIBE() TELLS YOU

- **count** — how many non-null values (missing data?)
- **mean** — average value (centre of the data)
- **std** — standard deviation (spread of the data)
- **min / max** — range (any extreme outliers?)

Why std matters: If mean rainfall is 6.2 mm but std is 15.8 mm, the data is highly skewed. Most days have zero rainfall, but when it rains, it pours.

DATA TYPES IN POLARS

CHECKING COLUMN TYPES

```
# Schema shows every column's type
print(df.schema)
# {'date': String,
#  'station': String,
#  'rainfall_mm': Float64,
#  'temp_max': Float64,
#  'temp_min': Float64,
#  'humidity': Int64}

# Single column access
print(df["rainfall_mm"].dtype)
# Float64
```

POLARS TYPE SYSTEM

Polars Type	Python Type	Example
Int64	int	Row counts
Float64	float	Prices, ratios
Utf8 / String	str	Town names
Boolean	bool	Flags
Date	date	Transaction dates

Common Mistake: Dates loaded as strings. Always check `df.schema` and cast with `str.to_date()`.

EXERCISE 1.1: SINGAPORE WEATHER EXPLORATION

YOUR TASK

```
import polars as pl
from shared import MLFPDataLoader

loader = MLFPDataLoader()
df = loader.load("mlfp01", "singapore_weather.csv")

# 1. Print the shape
print(f"Rows: {df.shape[0]}, Cols: {df.shape[1]}")

# 2. Print all column names
print(f"Columns: {df.columns}")

# 3. Show first 5 rows
print(df.head())

# 4. Run describe and answer:
stats = df.describe()
print(stats)
```

ANSWER THESE QUESTIONS

1. How many days of data do we have?
2. What is the hottest temperature ever recorded?
3. What is the average daily rainfall in mm?

Assessment: Code runs without error. All 3 questions answered correctly with evidence from `describe()` output.

LESSON 1.1 RECAP

PYTHON FUNDAMENTALS

- **Variables** — store values with `=`
- **Data types** — `int`, `float`, `str`, `bool`
- **Arithmetic** — `+` `-` `*` `/` `//` `**` `%`
- **F-strings** — `f"Value: {x:,.2f}"`

POLARS ESSENTIALS

- `pl.read_csv()` — load data
- `df.shape` — (rows, columns)
- `df.columns` — column names
- `df.head()` — first N rows
- `df.describe()` — summary statistics
- `df.schema` — column types

LESSON 1.2: FILTERING AND TRANSFORMING DATA

BOOLEANS, EXPRESSIONS, AND METHOD CHAINING

WHAT YOU WILL LEARN

- Boolean values and comparison operators
- `pl.col()` expressions for column access
- `filter()`, `select()`, `sort()`
- `with_columns()` for computed columns
- Method chaining (fluent API style)

Prerequisite: Lesson 1.1 (variables, DataFrame basics)

Note: You are writing *expressions* that evaluate to True/False. Python also has `if/else` for code-level decisions — that comes in Lesson 1.4.

BOOLEANS AND COMPARISONS

COMPARISON OPERATORS

```
price = 550000

# Comparisons return True or False
print(price > 500000) # True
print(price == 600000) # False
print(price != 550000) # False
print(price >= 550000) # True
print(price < 400000) # False
```

COMBINING CONDITIONS

```
# & means AND, | means OR
is_expensive = price > 500000
is_large = area > 100

both = is_expensive & is_large # True only if BOTH
either = is_expensive | is_large # True if EITHER
```

OPERATOR REFERENCE

Operator	Meaning
=	Equal to
≠	Not equal to
>	Greater than
<	Less than
≥	Greater or equal
≤	Less or equal
&	AND (both true)
	OR (either true)

Common Mistake: Using = (assignment) instead of == (comparison).

FILTERING ROWS WITH FILTER()

```
import polars as pl

df = pl.read_csv("data/hdb_resale.csv")

# Filter: only Queenstown flats
queenstown = df.filter(pl.col("town") == "QUEENSTOWN")
print(f"Queenstown transactions: {queenstown.shape[0]:,}")

# Filter: expensive flats (over $700K)
expensive = df.filter(pl.col("resale_price") > 700_000)
print(f"Over $700K: {expensive.shape[0]:,}")

# Combine conditions: Queenstown AND over $700K
qt_expensive = df.filter(
    (pl.col("town") == "QUEENSTOWN") &
    (pl.col("resale_price") > 700_000)
)
print(f"Queenstown over $700K: {qt_expensive.shape[0]:,}")
```

Key Pattern: `df.filter(pl.col("column") operator value)` — this is the syntax you will use hundreds of times.

SELECTING COLUMNS AND SORTING

SELECT() — CHOOSE COLUMNS

```
# Select specific columns
subset = df.select(
    "town", "flat_type",
    "floor_area_sqm", "resale_price"
)
print(subset.head())

# Select with expressions
subset = df.select(
    pl.col("town"),
    pl.col("resale_price") / 1000
)
```

SORT() — ORDER ROWS

```
# Sort by price (ascending)
sorted_df = df.sort("resale_price")

# Sort descending
sorted_df = df.sort("resale_price",
                    descending=True)

# Sort by multiple columns
sorted_df = df.sort(
    "town", "resale_price",
    descending=[False, True]
)
```

CREATING COMPUTED COLUMNS

```
# with_columns adds new columns (or overwrites existing ones)
df = df.with_columns(
    # Price per square metre
    (pl.col("resale_price") / pl.col("floor_area_sqm"))
    .alias("price_per_sqm"),

    # Remaining lease in years (from months)
    (pl.col("remaining_lease_months") / 12)
    .round(1)
    .alias("remaining_lease_years"),

    # High-value flag
    (pl.col("resale_price") > 600_000)
    .alias("is_high_value")
)

print(df.select("town", "price_per_sqm", "remaining_lease_years", "is_high_value").head())
```

Why expressions, not loops? Polars expressions run on the entire column at once (vectorised). Processing 15 million rows takes milliseconds, not minutes. This is why Polars beats pandas for large datasets.

METHOD CHAINING

CHAINING = READABILITY

```
# Instead of 5 separate variables...
result = (
    df
    .filter(pl.col("town") == "TAMPINES")
    .with_columns(
        (pl.col("resale_price") /
         pl.col("floor_area_sqm"))
        .alias("price_per_sqm")
    )
    .select(
        "month", "flat_type",
        "resale_price", "price_per_sqm"
    )
    .sort("price_per_sqm",
          descending=True)
    .head(10)
)
print(result)
```

THE PATTERN

Each method returns a new DataFrame, so you can chain the next method directly:

1. **filter** — select rows
2. **with_columns** — compute new columns
3. **select** — choose columns
4. **sort** — order results
5. **head** — limit output

Tip: Wrap the chain in parentheses () so you can split across lines without backslashes.

EXERCISE 1.2: HDB RESALE FILTERING

YOUR TASK

```
import polars as pl
from shared import MLFPDataLoader

loader = MLFPDataLoader()
df = loader.load("mlfp01", "hdb_resale.csv")

# 1. Filter by town = "BISHAN"
# 2. Filter by price 400K-800K
# 3. Create price_per_sqm column
# 4. Sort by price_per_sqm descending
# 5. Show top 10 results
```

EXPECTED OUTPUT

A DataFrame with 10 rows showing the most expensive-per-sqm Bishan flats in the \$400K-\$800K range.

Assessment: Correct filters applied. New column computed correctly. Output sorted by price_per_sqm descending.

LESSON 1.2 RECAP

BOOLEAN LOGIC

- Comparison: `=` `≠` `>` `<` `≥` `≤`
- Combine: `&` (AND), `|` (OR)
- Always parenthesise conditions

POLARS OPERATIONS

- `df.filter()` — select rows by condition
- `df.select()` — choose columns
- `df.sort()` — order rows
- `df.with_columns()` — add computed columns
- `.alias()` — name the result
- **Method chaining** — pipe operations together

LESSON 1.3: FUNCTIONS AND AGGREGATION

REUSABLE CODE AND GROUP-BY OPERATIONS

- Define functions with `def`, parameters, `return`
- Use `for` loops and collections (lists, dicts)
- Aggregate data with `group_by()` and `agg()`
- Write reusable helper functions for analysis

Prerequisite: Lesson 1.2 (filtering, expressions)

Spectrum Position: Data summarisation — compressing millions of rows into actionable statistics.

DEFINING FUNCTIONS

```
def price_per_sqm(price, area):  
    """Calculate price per square metre."""  
    return price / area  
  
result = price_per_sqm(550000, 92)  
print(f"${result:,.0f} per sqm")  
# Output: $5,978 per sqm  
  
def format_price(amount, currency="$"):  
    """Format a price with commas."""  
    return f"{currency}{amount:,.0f}"  
  
print(format_price(550000))          # $550,000  
print(format_price(550000, "SGD ")) # SGD 550,000
```

ANATOMY OF A FUNCTION

- **def** — declares a new function
- **Name** — descriptive, lowercase, underscores
- **Parameters** — inputs in parentheses
- **Docstring** — describes what it does
- **return** — sends back the result

Rule of Thumb: If you write the same code more than twice, wrap it in a function.

LISTS, DICTIONARIES, AND LOOPS

COLLECTIONS

```
towns = ["Queenstown", "Bishan",  
         "Tampines", "Ang Mo Kio"]  
print(towns[0])      # "Queenstown"  
print(len(towns))    # 4  
  
town_prices = {  
    "Queenstown": 580000,  
    "Bishan": 650000,  
    "Tampines": 420000,  
}  
print(town_prices["Bishan"]) # 650000
```

FOR LOOPS

```
for town in towns:  
    print(f"Processing {town}...")  
  
for town, price in town_prices.items():  
    psm = price / 92  
    print(f"{town}: ${psm:,.0f}/sqm")  
# Queenstown: $6,304/sqm  
# Bishan: $7,065/sqm  
# Tampines: $4,565/sqm
```

Note: Loops are for Python logic. For column-level operations, prefer Polars expressions (vectorised, much faster).

GROUP-BY AND AGGREGATION

```
town_stats = (  
    df  
    .group_by("town")  
    .agg(  
        pl.col("resale_price").mean().alias("avg_price"),  
        pl.col("resale_price").median().alias("median_price"),  
        pl.col("resale_price").count().alias("num_transactions"),  
        pl.col("resale_price").std().alias("price_std"),  
    )  
    .sort("avg_price", descending=True)  
)  
print(town_stats.head(10))
```

What group_by does: It splits the data into groups (one per unique town), applies the aggregation functions to each group independently, then combines the results. SQL users: this is `SELECT town, AVG(resale_price) FROM hdb GROUP BY town`.

AGGREGATION FUNCTIONS

Function	Description
<code>pl.mean()</code>	Average
<code>pl.median()</code>	Middle value
<code>pl.sum()</code>	Total
<code>pl.count()</code>	Number of rows
<code>pl.min()</code>	Minimum
<code>pl.max()</code>	Maximum
<code>pl.std()</code>	Standard deviation
<code>pl.first()</code>	First value
<code>pl.last()</code>	Last value
<code>pl.n_unique()</code>	Distinct count

MULTIPLE GROUPING COLUMNS

```

detailed = (
    df
    .group_by("town", "flat_type")
    .agg(
        pl.col("resale_price").mean()
        .alias("avg_price"),
        pl.count().alias("n"),
    )
    .sort("town", "flat_type")
)

print(df.select(
    pl.col("town").n_unique()
))

```

REUSABLE ANALYSIS FUNCTIONS

```
def district_summary(df: pl.DataFrame, district: str) → pl.DataFrame:
    """Compute summary statistics for a given district."""
    return (
        df
        .filter(pl.col("town") == district)
        .group_by("flat_type")
        .agg(
            pl.col("resale_price").mean().alias("avg_price"),
            pl.col("resale_price").count().alias("transactions"),
            (pl.col("resale_price").max() - pl.col("resale_price").min()).alias("price_range"),
        )
        .sort("avg_price", descending=True)
    )

print(district_summary(df, "QUEENSTOWN"))
print(district_summary(df, "BISHAN"))
print(district_summary(df, "ANG MO KIO"))
```

The Power of Functions: Write once, call for any district. Change the logic in one place and every call benefits.

LOOPING OVER DISTRICTS

BUILDING A REPORT

```
# Get unique towns
towns = df.select(
    pl.col("town").unique()
).to_series().to_list()

# Loop and build summaries
results = []
for town in towns[:5]: # top 5
    summary = district_summary(df, town)
    avg = summary["avg_price"][0]
    results.append({"town": town, "avg": avg})
    print(f"{town}: ${avg:,.0f}")

# Convert to DataFrame
report = pl.DataFrame(results)
print(report.sort("avg", descending=True))
```

PATTERN: LOOP + FUNCTION + COLLECT

1. Get the list of unique values
2. Loop over each value
3. Call your function for each
4. Collect results into a list
5. Convert to DataFrame for further analysis

Performance Note: This loop calls the function 5 times. For a full analysis, a single `group_by` is much faster. Use loops for formatted reports, `group_by` for computation.

EXERCISE 1.3: DISTRICT-LEVEL STATISTICS

YOUR TASK

Write functions to compute mean price, transaction count, and price range per district. Use `group_by` + `agg` for a summary table of ALL districts. Bonus: loop over top 5 and print a formatted report.

Assessment: Functions are reusable (not hardcoded to specific districts). Aggregation produces correct grouped results.

LESSON 1.3 RECAP

PYTHON FUNCTIONS

- `def name(params): return value`
- Default parameters: `def f(x, y=10)`
- Docstrings: `"""Description"""`
- Lists: `[1, 2, 3]`, Dicts: `{"k": "v"}`
- For loops: `for item in collection:`

POLARS AGGREGATION

- `group_by("col").agg(...)`
- `pl.mean()`, `pl.sum()`, `pl.count()`
- `pl.min()`, `pl.max()`, `pl.std()`
- `pl.n_unique()` for cardinality
- Multiple grouping columns supported

LESSON 1.4: JOINS AND MULTI-TABLE DATA

CONDITIONALS, IMPORTS, AND RELATIONAL JOINS

- `if/else/elif` conditional statements
- `import` and external packages
- Join concepts: left, inner, outer
- Polars `join()` on HDB + MRT + school data
- Dictionary lookups and mapping

Prerequisite: Lesson 1.3 (functions, collections, aggregation)

Spectrum Position: Data integration — combining information from multiple sources.

CONDITIONAL STATEMENTS

IF / ELIF / ELSE

```
price = 750000

if price > 1_000_000:
    category = "Million-dollar"
elif price > 700_000:
    category = "Premium"
elif price > 500_000:
    category = "Mid-range"
else:
    category = "Affordable"

print(f"${price:,} is {category}")
# Output: $750,000 is Premium
```

USING CONDITIONALS IN FUNCTIONS

```
def classify_flat(price, area):
    """Classify flat by price-per-sqm."""
    psm = price / area
    if psm > 8000:
        return "Luxury"
    elif psm > 6000:
        return "Premium"
    elif psm > 4000:
        return "Standard"
    else:
        return "Value"

print(classify_flat(750000, 92))
# Output: Premium
```

IMPORTS AND PACKAGES

```
import polars as pl
import json
from pathlib import Path
from datetime import datetime, timedelta

today = datetime.now()
one_year_ago = today - timedelta(days=365)
print(f"One year ago: {one_year_ago:%Y-%m-%d}")

config_path = Path("config.json")
with open(config_path) as f:
    config = json.load(f)
```

PACKAGES YOU WILL USE IN MLFP

Package	Purpose
polars	DataFrames
plotly	Visualisation
kailash_ml	ML engines
json	JSON parsing
pathlib	File paths
datetime	Dates and times
dotenv	Environment vars

VECTORISED CONDITIONALS IN POLARS

PYTHON IF/ELSE VS POLARS WHEN/THEN

```
# Python if/else: works on ONE value
def classify(price):
    if price > 1_000_000:
        return "Million"
    else:
        return "Standard"

# Polars when/then: works on ENTIRE column
df = df.with_columns(
    pl.when(pl.col("resale_price") > 1_000_000)
      .then(pl.lit("Million"))
      .when(pl.col("resale_price") > 700_000)
      .then(pl.lit("Premium"))
      .otherwise(pl.lit("Standard"))
      .alias("price_category")
)
```

WHY WHEN/THEN?

- **Speed:** Processes millions of rows in milliseconds
- **No loops:** The entire column is categorised at once
- **Chainable:** Multiple conditions without nested if/else

Rule: Use Python if/else for code-level decisions (which function to call). Use Polars when/then for data-level decisions (categorise rows).

JOIN CONCEPTS

Type	Keeps	Use When
Inner	Only matching rows	Need complete data only
Left	All left + matching right	Keep all primary records
Outer	All rows from both	Want everything, fill nulls

```
# HDB data has: town, price, area
# MRT data has: town, nearest_mrt, distance_km
```

```
# Inner: only towns in BOTH datasets
# Left:  all HDB towns, nulls if no MRT
# Outer: all towns from both sources
```

Key Decision: Do you want to *drop* rows without a match (inner) or *keep* them with nulls (left/outer)?

POLARS JOINS IN PRACTICE

```
hdb = pl.read_csv("data/hdb_resale.csv")
mrt = pl.read_csv("data/mrt_stations.csv")
schools = pl.read_csv("data/primary_schools.csv")

# Left join: keep all HDB records
hdb_mrt = hdb.join(mrt, on="town", how="left")
print(f"After MRT join: {hdb_mrt.shape}")

# Chain a second join: add school proximity
hdb_enriched = (
    hdb
    .join(mrt, on="town", how="left")
    .join(schools, on="town", how="left")
)
print(f"Enriched: {hdb_enriched.shape}")
```

Pattern: `left_df.join(right_df, on="key_column", how="left")` — always specify `how=` explicitly.

HANDLING MISSING JOINS

```
null_counts = hdb_enriched.null_count()
print(null_counts)

no_mrt = hdb_enriched.filter(
    pl.col("nearest_mrt").is_null()
)
print(f"Missing MRT: {no_mrt.shape[0]:,}")

region_map = {
    "QUEENSTOWN": "Central",
    "BISHAN": "Central",
    "TAMPINES": "East",
    "WOODLANDS": "North",
}
hdb_enriched = hdb_enriched.with_columns(
    pl.col("town").replace(region_map)
    .alias("region")
)
```

Inner join: Use when you only want complete data. Unmatched rows are dropped.

Left join: Keep all primary records. Missing data appears as null. Always check null counts.

Row count check: After a left join, result rows should equal left table rows (unless duplicates on the join key).

EXERCISE 1.4: MULTI-TABLE HDB ANALYSIS

1. Load HDB resale, MRT station, and school data
2. Join all three on the appropriate key
3. Compute distance-to-amenity features
4. Handle missing joins appropriately
5. Verify the combined dataset row count

Assessment: Correct join type selected. Missing data handled (not silently dropped). Combined dataset has expected row count.

LESSON 1.4 RECAP

- `if / elif / else` for branching
- `import package` and `from X import Y`
- Dictionary-based value mapping
- `df.join(other, on="key", how="left")`
- **Inner:** only matching rows
- **Left:** all left + matching right
- **Outer:** all from both sides
- Always check `null_count()` after joins

LESSON 1.5: WINDOW FUNCTIONS AND TRENDS

ROLLING STATISTICS, YEAR-ON-YEAR CHANGES, AND LAZY FRAMES

- Window functions with `over()`
- Rolling averages with `rolling_mean()`
- Shifted values with `shift()`
- Year-on-year calculations
- Lazy frames for performance

Prerequisite: Lesson 1.4 (joins, conditionals)

Spectrum Position: Temporal feature creation — extracting time-based patterns.

WINDOW FUNCTIONS WITH OVER()

GROUP_BY VS OVER()

```
# group_by: one row per group
town_avg = df.group_by("town").agg(
    pl.col("resale_price").mean()
    .alias("avg_price")
) # 26 rows

# over(): result on EVERY row
df_with_avg = df.with_columns(
    pl.col("resale_price").mean()
    .over("town")
    .alias("town_avg_price")
) # same rows as original
```

COMPARE EACH FLAT TO ITS TOWN

```
df_with_avg = df.with_columns(
    pl.col("resale_price").mean()
    .over("town")
    .alias("town_avg"),
).with_columns(
    (pl.col("resale_price") -
     pl.col("town_avg"))
    .alias("price_vs_avg")
)
# Positive = above town average
# Negative = below town average
```

SQL Equivalent:

AVG(price) OVER (PARTITION BY town)

ROLLING AVERAGES

```
monthly = (  
    df  
    .group_by("town", "month")  
    .agg(pl.col("resale_price").mean().alias("avg_price"))  
    .sort("town", "month")  
)  
  
# 3-month rolling average per town  
monthly = monthly.with_columns(  
    pl.col("avg_price")  
    .rolling_mean(window_size=3)  
    .over("town")  
    .alias("rolling_3m_avg")  
)
```

Rolling Window: For each row, the rolling average looks at the current row and the previous N-1 rows. With `window_size=3`, it averages the current month and two preceding months. The first two rows will be null (not enough history).

YEAR-ON-YEAR CHANGES WITH SHIFT()

```
monthly = monthly.with_columns(  
    pl.col("avg_price")  
        .shift(12)  
        .over("town")  
        .alias("price_12m_ago")  
)  
  
monthly = monthly.with_columns(  
    ((pl.col("avg_price") -  
      pl.col("price_12m_ago")) /  
     pl.col("price_12m_ago") * 100)  
    .round(1)  
    .alias("yoy_change_pct")  
)
```

```
growth = (  
    monthly  
    .filter(pl.col("yoy_change_pct").is_not_null())  
    .group_by("town")  
    .agg(  
        pl.col("yoy_change_pct").mean()  
        .alias("avg_yoy_growth")  
    )  
    .sort("avg_yoy_growth", descending=True)  
)  
print(growth.head(5))
```

YoY > 0: Prices rose vs last year

YoY < 0: Prices fell vs last year

IDENTIFYING TRENDS AND SEASONALITY

SINGAPORE PROPERTY SEASONS

```
# Extract month number
monthly = monthly.with_columns(
    pl.col("month").str.slice(5, 2)
    .cast(pl.Int32)
    .alias("month_num")
)

# Average price by calendar month
seasonal = (
    monthly
    .group_by("month_num")
    .agg(pl.col("avg_price").mean()
        .alias("seasonal_avg"))
    .sort("month_num")
)

print(seasonal)
```

WHAT TO LOOK FOR

- **Trend:** Long-term direction (up or down over years)
- **Seasonality:** Repeating pattern within each year (e.g., CNY dip in Feb)
- **Cyclicity:** Multi-year patterns (property cooling measures)
- **Noise:** Random month-to-month variation

Singapore Context: HDB resale volumes typically dip in February (Chinese New Year) and spike in July-August (school-year timing for families with children).

LAZY FRAMES: MAKE IT FASTER

```
# Eager: executes immediately
df = pl.read_csv("data/hdb_resale.csv")
result = (
    df
    .filter(pl.col("town") == "BISHAN")
    .select("month", "resale_price")
)

# Lazy: builds query plan, executes on collect()
lf = pl.scan_csv("data/hdb_resale.csv")
result = (
    lf
    .filter(pl.col("town") == "BISHAN")
    .select("month", "resale_price")
    .collect()
)
```

WHY LAZY?

- **Predicate pushdown:** filter before loading all columns
- **Projection pushdown:** only read selected columns
- **Query optimisation:** Polars reorders operations

When to use: Large files (>1M rows), multiple chained operations. Not required for small datasets.

INSPECTING THE QUERY PLAN

```
lf = (  
    pl.scan_csv("data/hdb_resale.csv")  
    .filter(pl.col("town") == "BISHAN")  
    .select("month", "resale_price")  
    .sort("month")  
)  
  
print(lf.explain())  
# SORT BY [col("month")]  
#   FAST_PROJECT: [month, resale_price]  
#     FILTER col("town") == "BISHAN"  
#       CSV SCAN data/hdb_resale.csv  
#       PROJECT 3/12 COLUMNS ← only reads 3 columns!
```

Performance: "PROJECT 3/12 COLUMNS" means Polars only reads 3 of the 12 columns from disk. For large files, lazy can be 2-5x faster than eager.

EXERCISE 1.5: HDB PRICE TRENDS

1. Compute monthly average prices per district
2. Add a 3-month rolling average per district
3. Calculate YoY price change percentage
4. Identify top 5 highest-growth districts
5. Identify top 5 lowest-growth districts

Assessment: Rolling calculations correct. YoY computed with proper time alignment. Growth ranking is accurate.

LESSON 1.5 RECAP

WINDOW FUNCTIONS

- `.over("col")` — compute per group, keep all rows
- `.rolling_mean(N)` — N-period moving average
- `.shift(N)` — look back N periods
- $\text{YoY} = (\text{current} - \text{prior}) / \text{prior} * 100$

LAZY EVALUATION

- `pl.scan_csv()` — lazy load
- `.collect()` — trigger execution
- `.explain()` — inspect query plan
- Optional: not needed for correctness

LESSON 1.6: DATA VISUALISATION

PRINCIPLES, CHART SELECTION, AND INTERACTIVE CHARTS

- Why visualise (tables vs charts)
- Attributes of good charts
- Gestalt principles for readability
- Chart selection by data type
- Plotly Express and ModelVisualizer

Prerequisite: Lesson 1.5 (aggregation, trends)

Spectrum Position: Data communication — making patterns visible.

WHY VISUALISE?

TABLES TELL, CHARTS REVEAL

- **Tables** show precise values — good for lookup
- **Charts** show patterns — good for discovery
- Anscombe's Quartet: 4 datasets, identical stats, vastly different shapes
- A histogram takes 1 second to spot bimodality. A table cannot.

Remember the flash crash? The dashboard showed a single median number. A histogram would have revealed the bimodal distribution instantly.

Rule: Always plot your data before computing statistics.

ATTRIBUTES OF GOOD CHARTS

FOUR QUALITIES

- **Simple** — one chart, one message
- **Clean** — no chart junk, no 3D effects
- **Subtle attention** — colour draws the eye to what matters
- **Truthful** — axes start at zero for bar charts

CHARTS TO AVOID

3D Charts: Distort proportions. Never use 3D for quantitative data.

Misleading Pie Charts: Use horizontal bar charts for >3 categories.

Dual Y-Axes: Create false correlations. Use two separate charts.

GESTALT PRINCIPLES

Principle	Application
Proximity	Items close together form a group
Similarity	Same colour = same group
Closure	We complete incomplete shapes
Enclosure	Borders and backgrounds group items
Continuity	We follow lines (line charts work)
Connection	Connected points feel related

VISUAL ORDER: Z-PATTERN

- **Top-left:** Title / key message
- **Top-right:** Legend / context
- **Bottom-left:** Axis labels
- **Bottom-right:** Source / notes

Tip: Title should state the insight: "Queenstown prices rose 12% YoY" beats "HDB Prices by Town".

CHART SELECTION GUIDE

Data Question	Chart Type	Example
How are values distributed?	Histogram	HDB price distribution
How does a value change over time?	Line chart	Monthly median price
How do categories compare?	Vertical bar	Average price by town
How do many categories rank?	Horizontal bar	26 towns ranked
What is the composition?	Stacked bar	Flat type mix per town
How do two variables relate?	Scatter plot	Area vs price
How do variables correlate?	Heatmap	Correlation matrix
Percentage breakdown?	100% stacked bar	Survey responses

PLOTLY EXPRESS: INTERACTIVE CHARTS

```
import plotly.express as px

fig = px.histogram(
    df.to_pandas(),
    x="resale_price", nbins=50,
    title="HDB Resale Price Distribution"
)
fig.show()

fig = px.line(
    monthly.to_pandas(),
    x="month", y="avg_price",
    color="town",
    title="Monthly Average Price by Town"
)
fig.show()
```

```
fig = px.bar(
    town_stats.to_pandas(),
    x="town", y="avg_price",
    title="Average Price by Town"
)
fig.show()

fig = px.scatter(
    df.to_pandas(),
    x="floor_area_sqm",
    y="resale_price",
    color="flat_type",
    title="Floor Area vs Price"
)
fig.show()
```

Note: Plotly requires pandas. Use `.to_pandas()` to convert from Polars. This is the only place we use pandas.

KAILASH MODELVISUALIZER

MODELVISUALIZER — ONE-LINE EDA CHARTS

```
from kailash_ml import ModelVisualizer

viz = ModelVisualizer()

viz.plot_correlation(df,
                    title="HDB Correlations")

viz.plot_distribution(df,
                    column="resale_price",
                    title="Price Distribution")
```

- Consistent styling across all MLFP modules
- Built-in best practices (no misleading axes)
- Polars-native: no `.to_pandas()` needed
- Extends into ML visualisations in later modules

Pattern: Plotly for custom charts. ModelVisualizer for standard EDA.

HEATMAPS AND STACKED BARS

```
numeric_cols = df.select(
    pl.col(pl.Float64, pl.Int64)
)
corr = numeric_cols.to_pandas().corr()

fig = px.imshow(
    corr, text_auto=".2f",
    color_continuous_scale="RdBu_r",
    title="HDB Feature Correlations"
)
fig.show()
```

```
composition = (
    df
    .group_by("town", "flat_type")
    .agg(pl.count().alias("n"))
)

fig = px.bar(
    composition.to_pandas(),
    x="town", y="n",
    color="flat_type",
    title="Flat Type Mix by Town",
    barmode="stack"
)
fig.show()
```

EXERCISE 1.6: SIX CHARTS FROM HDB DATA

1. **Heatmap** — price feature correlations
2. **Line chart** — monthly price trend for 3 towns
3. **Bar chart** — top 10 towns by average price
4. **Scatter plot** — floor area vs resale price
5. **Histogram** — price distribution
6. **Stacked bar** — flat type composition by town

Assessment: All 6 charts created. Chart type matches data question. No misleading axes. Every chart has a descriptive title. Interactive features work.

LESSON 1.6 RECAP

DESIGN PRINCIPLES

- Simple, clean, subtle, truthful
- Gestalt: proximity, similarity, closure
- Z-pattern reading order
- Avoid: 3D, misleading pie, dual Y-axes

TECHNICAL TOOLS

- `px.histogram()`, `px.line()`
- `px.bar()`, `px.scatter()`
- `px.imshow()` for heatmaps
- `ModelVisualizer` for one-line EDA

LESSON 1.7: AUTOMATED DATA PROFILING

DATAEXPLORER, ALERTCONFIG, AND ERROR HANDLING

- DataExplorer: profiling with 8 alert types
- AlertConfig: configure quality thresholds
- DataProfile: access results programmatically
- `compare()` : compare two datasets
- `try/except` for error handling

Prerequisite: Lesson 1.6 (visualisation)

Spectrum Position: Automated data assessment — machine-detected quality issues.

DATAEXPLORER: ONE-CALL PROFILING

DATAEXPLORER — AUTOMATED DATA QUALITY ASSESSMENT

```
from kailash_ml import DataExplorer
from shared import run_profile

explorer = DataExplorer()
profile = run_profile(explorer, df)

print(f"Rows: {profile.row_count}")
print(f"Columns: {profile.column_count}")
print(f"Missing: {profile.missing_summary}")
print(f"Alerts: {len(profile.alerts)}")

for alert in profile.alerts:
    print(f"    [{alert.severity}] {alert.column}: {alert.message}")
```

Checks: Missing values, outliers, duplicates, skewness, high correlation, low cardinality, constant columns, type inference issues.

ALERTCONFIG: CUSTOM THRESHOLDS

```
from kailash_ml import AlertConfig

config = AlertConfig(
    missing_threshold=0.05,
    outlier_std=3.0,
    duplicate_threshold=0.01,
    skew_threshold=2.0,
    correlation_threshold=0.95,
    cardinality_threshold=0.95,
    constant_threshold=0.99,
)

explorer = DataExplorer(alert_config=config)
profile = run_profile(explorer, df)
```

8 ALERT TYPES

Alert	Detects
Missing	Null values above threshold
Outlier	Extreme values (by std)
Duplicate	Repeated rows
Skewness	Non-symmetric distributions
Correlation	Redundant features
Cardinality	Near-unique columns
Constant	No-variance columns
Type	Type inference issues

COMPARING DATASETS

```
profile_before = run_profile(explorer, df_dirty)

df_clean = (
    df_dirty
    .drop_nulls(subset=["resale_price"])
    .filter(pl.col("resale_price") > 0)
)

profile_after = run_profile(explorer, df_clean)

comparison = explorer.compare(
    profile_before, profile_after
)
print(comparison.summary)
```

WHAT COMPARE() SHOWS

- Row count change
- Alert count change
- Distribution shifts per column
- Missing value reduction
- Outlier count change

Use Case: Compare train vs test for drift. Compare pre vs post cleaning.

ERROR HANDLING WITH TRY/EXCEPT

```
try:
    df = pl.read_csv("data/economic.csv")
    profile = run_profile(explorer, df)
    print(f"Profiled {df.shape[0]} rows")
except FileNotFoundError:
    print("Error: Data file not found.")
except Exception as e:
    print(f"Unexpected error: {e}")
finally:
    print("Profiling attempt complete.")
```

WHEN TO USE

- **File operations** — file may not exist
- **API calls** — network may fail
- **Data parsing** — format may be wrong

Never do: `except: pass`
Silent failures are the hardest bugs to find.

EXERCISE 1.7: PROFILE AND COMPARE

1. Load the dirty economic indicators dataset
2. Profile with DataExplorer
3. Configure AlertConfig with sensible thresholds
4. Clean the data
5. Profile the cleaned version
6. Compare original vs cleaned

Assessment: All quality issues identified. Alerts configured with sensible thresholds. Comparison shows improvement.

LESSON 1.7 RECAP

DATAEXPLORER

- `DataExplorer()` + `run_profile()`
- `AlertConfig` for thresholds
- 8 alert types
- `compare()` for before vs after

ERROR HANDLING

- `try / except / finally`
- Catch specific exceptions
- Never bare `except: pass`

LESSON 1.8: DATA PIPELINES AND END-TO-END PROJECT

ETL, APIS, PREPROCESSING PIPELINE, AND PROJECT STRUCTURE

- Null handling: `is_null()`, `fill_null()`, `drop_nulls()`
- ETL concepts: Extract, Transform, Load
- REST APIs: GET, POST, JSON (OneMap Singapore)
- PreprocessingPipeline for automated cleaning
- Multi-file Python project structure

Prerequisite: All of M1 (Lessons 1.1-1.7)

Capstone: Build a complete pipeline: raw data to clean output with visualisations.

HANDLING NULL VALUES

DETECTION

```
print(df.null_count())

null_rows = df.filter(
    pl.col("resale_price").is_null()
)
print(f"Null prices: {null_rows.shape[0]}")

total = df.shape[0]
for col in df.columns:
    pct = df[col].null_count() / total * 100
    if pct > 0:
        print(f"{col}: {pct:.1f}% missing")
```

TREATMENT

```
df_clean = df.drop_nulls()

df_clean = df.drop_nulls(
    subset=["resale_price", "town"]
)

df_filled = df.with_columns(
    pl.col("floor_area_sqm")
        .fill_null(pl.col("floor_area_sqm").median())
)

df_filled = df.with_columns(
    pl.col("resale_price")
        .fill_null(strategy="forward")
)
```

REST APIS: EXTRACTING DATA

ONEMAP SINGAPORE EXAMPLE

```
import requests

url = "https://www.onemap.gov.sg/api/common/elastic/search"
params = {
    "searchVal": "Queenstown MRT",
    "returnCoord": "Y",
    "getAddrDetails": "Y",
}
response = requests.get(url, params=params)
data = response.json()
```

PARSING THE RESPONSE

```
results = data["results"]
first = results[0]
lat = float(first["LATITUDE"])
lng = float(first["LONGITUDE"])
print(f"({first['ADDRESS']}) : ({lat}, {lng})")

df_locations = pd.DataFrame(results)
```

```
try:
    response = requests.get(url, params=params)
    response.raise_for_status()
except requests.exceptions.HTTPError as e:
    print(f"API error: {e}")
```

KAILASH PREPROCESSING PIPELINE

PREPROCESSING PIPELINE — AUTOMATED DATA CLEANING

```
from kailash_ml import PreprocessingPipeline

pipeline = PreprocessingPipeline()
df_clean = pipeline.fit_transform(df)

print(pipeline.steps_applied)
# ['type_inference',
#  'missing_imputation',
#  'categorical_encoding',
#  'numeric_scaling',
#  'outlier_handling']
```

WHAT IT AUTOMATES

- **Type inference**
- **Missing imputation** (median/mode)
- **Categorical encoding**
- **Numeric scaling**
- **Outlier handling**

Why? Manual cleaning is error-prone and non-reproducible. The pipeline applies the same steps every time.

THE ETL PATTERN

```
# EXTRACT
df_hdb = pl.read_csv("data/hdb_realte.csv")
df_art = pl.read_csv("data/art_stations.csv")

# TRANSFORM
pipeline = PreprocessingPipeline()
df_clean = pipeline.fit_transform(df_hdb)
df_enriched = df_clean.join(
    df_art, on="town", how="left"
)

# LOAD
df_enriched.write_csv("output/hdb_clean.csv")
df_enriched.write_parquet("output/hdb_clean.parquet")
```

FULL PIPELINE STEPS

1. **Extract** — files, APIs, databases
2. **Profile** — DataExplorer
3. **Transform** — PreprocessingPipeline
4. **Enrich** — joins
5. **Visualise** — Plotly / ModelVisualizer
6. **Load** — CSV, Parquet, database

Parquet vs CSV: Parquet is binary, compressed, columnar. 5-10x smaller.

PROJECT STRUCTURE

```
project/  
  data/  
    raw/          # Never modify originals  
    processed/    # Cleaned data  
  src/  
    load.py       # Extract functions  
    clean.py      # Transform functions  
    visualise.py  # Chart functions  
    report.py     # Report generation  
  output/  
    charts/  
    report.html  
  main.py         # Orchestrates pipeline  
  requirements.txt
```

WHY STRUCTURE MATTERS

- **Separation:** data, code, output in separate folders
- **Immutable raw data:** never modify originals
- **Reproducibility:** `main.py` recreates everything

Rule: If you cannot delete `output/` and regenerate it by running `main.py`, your pipeline is broken.

EXERCISE 1.8: FULL EDA PIPELINE

1. **Extract:** Load messy taxi trip data
2. **Profile:** DataExplorer, document all issues
3. **Clean:** PreprocessingPipeline
4. **Visualise:** At least 3 charts with insights
5. **Report:** Generate HTML report

Assessment: Pipeline runs E2E. Data quality measurably improved. Report contains at least 3 visualisations with written insights.

CAPSTONE PIPELINE CODE

```
import pandas as pd
from kailash.ai import DataExplorer, PreprocessingPipeline, ModelVisualizer
from shared import MLFPDataLoader, run_profile
from utils import load_data

load_data()

# 1. EXTRACT
loader = MLFPDataLoader()
df = loader.load("n1tp01", "taxi.trips.csv")

# 2. PROFILE
explorer = DataExplorer()
profile_before = run_profile(explorer, df)

# 3. CLEAN
pipeline = PreprocessingPipeline()
df_clean = pipeline.fit_transform(df)
profile_after = run_profile(explorer, df_clean)
comparison = explorer.compare(profile_before, profile_after)

# 4. VISUALIZE
viz = ModelVisualizer()
viz.plot_distribution(df_clean, column="trip_distance", title="Trip Distance Distribution")
viz.plot_correlation(df_clean, title="Feature Correlations")

# 5. REPORT
print(comparison.summary)
```

LESSON 1.8 RECAP

DATA CLEANING

- `df.null_count()`, `drop_nulls()`, `fill_null()`
- `PreprocessingPipeline` for automation

PIPELINE ENGINEERING

- ETL: Extract, Transform, Load
- REST APIs: `requests.get()`
- Parquet for fast storage
- Project structure: `data/`, `src/`, `output/`
- Reproducibility via `main.py`

MODULE 1 SUMMARY

PYTHON SKILLS ACQUIRED

- Variables, data types, f-strings
- Arithmetic and comparisons
- Functions, parameters, return values
- Lists, dictionaries, for loops
- if/elif/else conditionals
- Imports and packages
- try/except error handling
- Project structure and modules

DATA SKILLS ACQUIRED

- Load, inspect, describe (Polars)
- Filter, select, sort, transform
- Group-by aggregation
- Multi-table joins
- Window functions, rolling averages
- Interactive visualisation (Plotly)
- Automated profiling (DataExplorer)
- Automated cleaning (PreprocessingPipeline)
- End-to-end ETL pipelines

You went from zero Python to building complete data pipelines in 8 lessons. In Module 2, you will use these skills to engineer features for machine learning models.